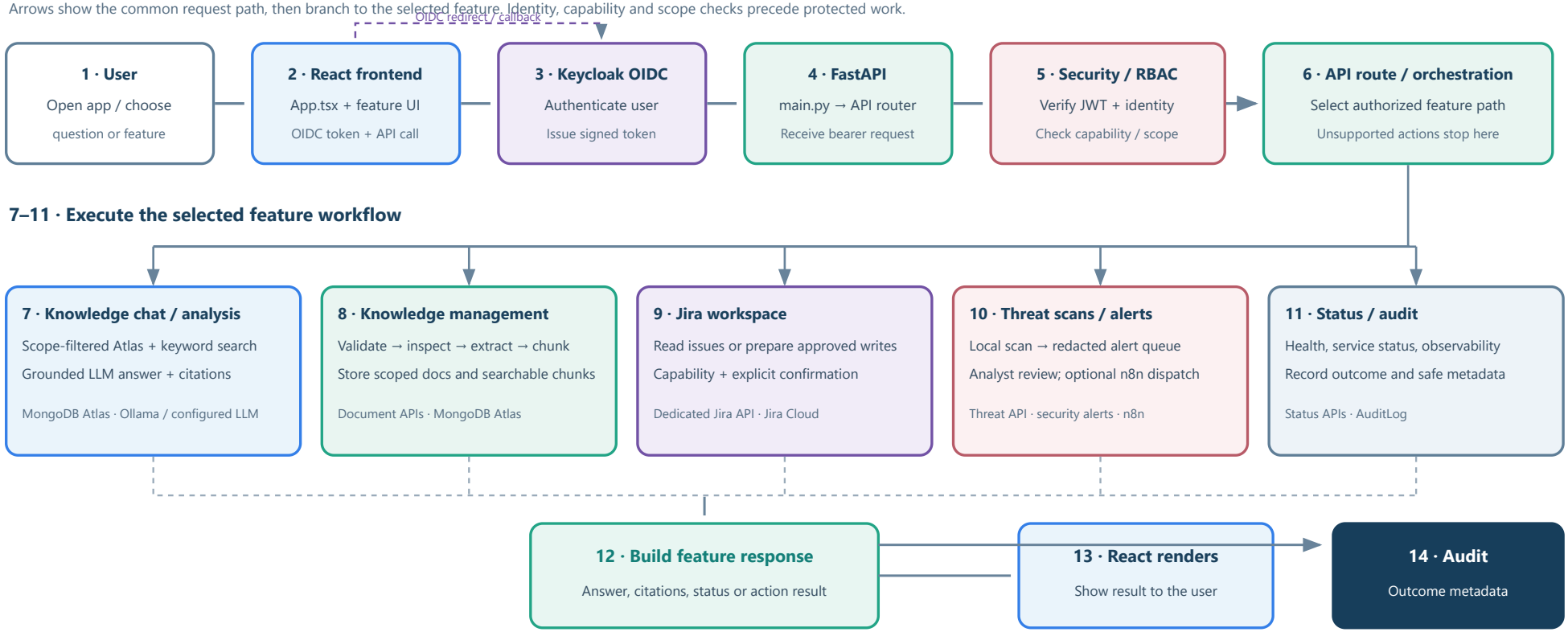
End-to-End System Workflow

One entry point, trusted access controls, feature-specific services, and a governed result

SCOPE Authentication · Knowledge chat · Knowledge management · Jira · Threat analysis · Security alerts · System status · Audit

KnowledgeOps AI · Whole Application

Arrows show the common request path, then branch to the selected feature. Identity, capability and scope checks precede protected work.



Branches are feature-specific: chat does not execute Jira or threat-response actions. Protected operations require their own capabilities and confirmations.

`frontend/src/app/App.tsx` — Application Shell & User State

This file coordinates the React workspace after the browser mounts the application. It connects the OIDC callback to the authenticated user profile and then chooses the sign-in or workspace view.

Core initialization excerpt

```
useEffect(() => {
  let active = true;
  const initialize = async () => {
    if (oidcMode) {
      await completeOidcCallback();
      if (!(await getOidcUser())) return;
      const currentUser = await getCurrentUser();
      if (active) setUser(currentUser);
    }
  };
  initialize().finally(() => {
    if (active) setLoading(false);
  });
  return () => { active = false; };
}, []);
```

What the file does

State — Stores user, activeView, loading, busy, and error state.

initialize() — Processes OIDC callback, checks for a valid OIDC user, calls `getCurrentUser()`, and stores the backend profile.

Login / logout — `handleLogin()` starts OIDC; `handleLogout()` clears workspace session state and starts provider logout.

Render — Shows loading, sign-in, or the workspace. Capability checks control navigation visibility.

Handoff: App.tsx calls `oidc.ts` for callback and sign-in state, then `auth.ts` for `GET /api/v1/me`. The returned backend user drives the workspace UI. Backend authorization remains authoritative.

`frontend/src/api/oidc.ts` — Keycloak OIDC Client

This file manages the browser-side sign-in redirect, callback, stored OIDC user, and access-token retrieval through `oidc-client-ts`.

Client configuration

```
manager = new UserManager({
  authority,
  client_id: clientId,
  redirect_uri: `${window.location.origin}/`,
  response_type: "code",
  scope: "openid profile email",
  automaticSilentRenew: true,
  userStore: new WebStorageStateStore({
    store: window.sessionStorage,
  }),
});
```

The browser holds OIDC state in session storage. The API still validates each bearer token itself.

Key functions and hand-off

userManager() — Creates and caches the OIDC manager from `VITE_OIDC_AUTHORITY` and `VITE_OIDC_CLIENT_ID`.

startOidcLogin() — Redirects the browser to Keycloak.

completeOidcCallback() — Processes returned code/state or provider error and removes callback query parameters.

getOidcUser() — Returns a stored user only if it exists and is not expired.

getOidcAccessToken() — Returns the current access token. `http.ts` adds it as `Authorization: Bearer …`; `auth.ts` calls `/api/v1/me`.

backend/app/core/auth.py — Bearer Verification & User Resolution

This file is the backend identity boundary. A browser-side signed-in state is not accepted as proof; the backend validates the token and constructs its own user context.

Token verification excerpt

```
signing_key = _jwks_client(jwks_url)
    .get_signing_key_from_jwt(token)
claims = jwt.decode(
    token,
    signing_key.key,
    algorithms=["RS256"],
    audience=audience,
    issuer=issuer,
    options={"require": ["exp", "iat", "sub", "iss", "aud"]},
)
```

Trust checks

Uses Keycloak's JWKS public key, then verifies signature, accepted algorithm, issuer, audience, expiry, and required claims.

Key functions

get_current_user() — FastAPI dependency. Missing/invalid bearer token is rejected.

authenticate_oidc_token() — Loads issuer/audience/JWKS config, validates token, reads subject and group claim.

_jwks_client() — Caches the PyJWT JWKS client for public signing keys.

_group_capability_mapping() — Reads configured group-to-capability policy and checks capabilities against the allowlist.

DemoUser result — Supplies user ID, display name, role, and server-derived capabilities to protected endpoints.

Input → output: `Authorization: Bearer <JWT>` plus backend OIDC settings → trusted `DemoUser`. Invalid/missing token returns 401; misconfigured OIDC returns 503. Group membership alone is not permission; capabilities come from backend configuration.

backend/app/api/v1/chat.py — Protected API Entry & RBAC Gate

This file defines the chat endpoint. It receives the request, asks FastAPI for a verified user, and checks the capability before processing a question.

Endpoint excerpt

```
@router.post("/chat", response_model=ChatResponse)
def chat(
    payload: ChatRequest,
    request: Request,
    user: Annotated[DemoUser,
        Depends(get_current_user)],
    index: Annotated[KnowledgeIndex,
        Depends(get_knowledge_index)],
) -> ChatResponse:
    require_capability(user, "chat:ask", request)
    input_decision = moderate_input(payload.question)
    ...
```

Route registration: `main.py → api/v1/router.py → chat.py`.

Key actions

Route + input — `POST /api/v1/chat`; ChatRequest validates question length and retrieval mode.

Identity + RBAC — `Depends(get_current_user)` resolves DemoUser; `require_capability()` from permissions.py requires `chat:ask` or denies with 403.

Processing hand-off — Input moderation runs before `route_question()` and `orchestrate_request()`.

Response + audit — Output moderation checks the answer; AuditLog records metadata; endpoint returns ChatResponse.

Input → output: `POST /api/v1/chat` with JSON and bearer token → ChatResponse. Identity failure is 401; missing `chat:ask` is 403; only an authorized request continues to moderation and downstream work.

KNOWLEDGEOPS AI | FINAL PAGE

Swimlane — Frontend → Identity → RBAC → API Entry

This swimlane follows the same four source files explained on pages 2–5. Other files appear only where they connect the hand-off.

START User opens the app · **END** Protected chat request is allowed into processing or rejected at the identity/capability gate



How to read it: follow the numbered steps and arrows across lanes. OIDC sign-in establishes the browser session; each API request carries a bearer token; backend token validation establishes identity; the capability check authorizes the specific operation.

Key distinction: hiding a frontend button is only a user-interface choice. The protected API's server-side identity and RBAC checks enforce access.